

Stateless and Stateful Services

Introduction

One of the most important concepts in scalable system design is understanding the difference between:

1. **Stateless Services**
2. **Stateful Services**

This directly affects:

- Scalability
- Availability
- Fault tolerance
- Session management
- Deployment strategy
- Load balancing
- Complexity

Core Idea

Stateless Service

Server does **not store client session/state** between requests.

Every request is independent.

Stateful Service

Server stores session or state information between requests.

Future requests may depend on previous interactions.

Why This Matters

Modern large-scale systems prefer **stateless application layers** because they are easier to scale horizontally.

1. Stateless Services

Definition

A stateless service does not remember previous client requests.

Each request contains everything needed to process it.

Example: HTTP APIs

HTTP is inherently stateless.

None

GET /products/101

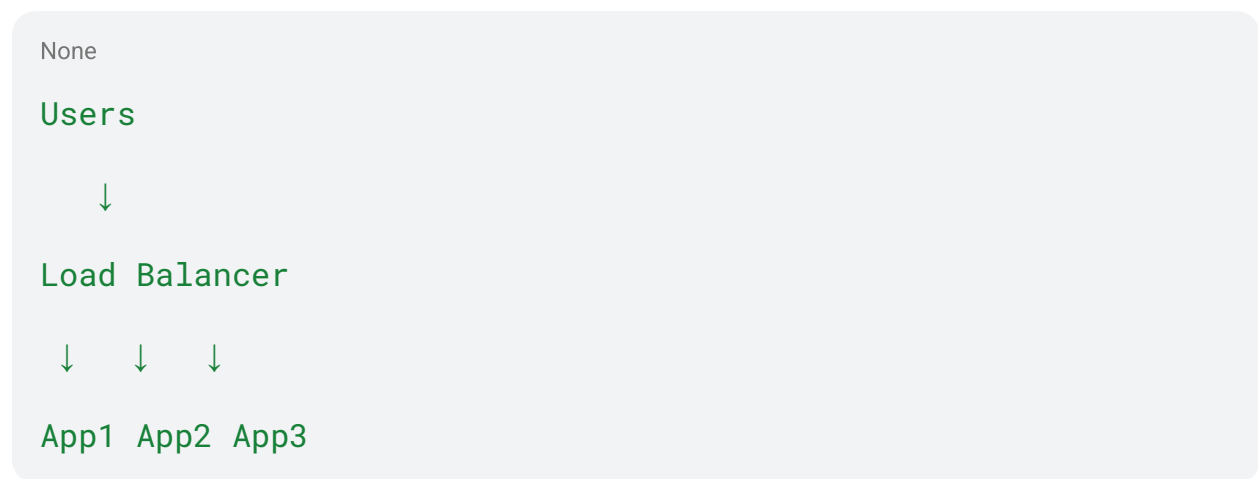
Authorization: Bearer token

Server processes request without needing memory of prior requests.

Examples of Stateless Systems

- REST APIs
- Web servers
- Microservice endpoints
- CDN edge servers
- Search APIs
- Auth token validation services

Example Architecture



Any request can go to any server.

Advantages of Stateless Services

1. Easy Horizontal Scaling

Just add more servers.

2. Better Load Balancing

Any instance can serve request.

3. Fault Tolerance

If one server dies, others continue.

4. Simpler Deployment

Rolling updates easier.

5. Auto Scaling Friendly

Cloud platforms prefer stateless services.

Disadvantages of Stateless Services

1. Need External State Storage

Sessions/data must go elsewhere:

- Database

- Cache
- Object storage

2. Slightly More Network Calls

Need to fetch user/session data externally.

Real Example

Netflix backend APIs

Requests routed to many stateless services.

2. Stateful Services

Definition

A stateful service stores client-specific state or application state locally.

Later requests depend on that stored state.

Examples

- Database servers
- Game servers
- WebSocket chat sessions
- In-memory shopping cart tied to one server
- FTP sessions
- Stateful firewalls

Example

User logs in.

Server stores:

None

`session_id -> user123`

`cart_items -> [phone, laptop]`

Next request must reach same server.

Stateful Architecture

None

`User A -> Server 1`

`User B -> Server 2`

Sticky sessions often required.

Advantages of Stateful Services

1. Faster Repeated Access

Local memory state is fast.

2. Rich Real-Time Interaction

Useful for gaming/chat/live collaboration.

3. Lower Repeated DB Reads

State already stored in memory.

Disadvantages of Stateful Services

1. Harder Horizontal Scaling

Need session affinity.

2. Failures Cause Session Loss

If server crashes, local state lost.

3. Complicated Deployment

Moving users between servers harder.

4. Uneven Load Distribution

Some nodes overloaded.

Comparison Table

Feature	Stateless	Stateful
Stores session locally	No	Yes
Easy scaling	High	Difficult
Load balancing	Easy	Complex
Failure recovery	Easier	Harder
Sticky sessions needed	No	Yes
Best for	APIs	DB/chat/games

Why HTTP Apps Prefer Stateless Design

HTTP request-response model naturally supports statelessness.

Each request carries:

- Headers
- Auth token
- Cookies
- Parameters
- Body

So any server can handle it.

How to Make Stateful Apps More Scalable

Move state outside app servers.

Common Pattern

None

User Request



Stateless App Server



Redis / DB stores session

Tools Used

- Redis for sessions
- Memcached
- PostgreSQL
- MongoDB

Session Management Approaches

1. Server-Side Session (Stateful)

Session stored on app node.

Need sticky sessions.

2. Shared Session Store

Session stored in Redis.

App becomes stateless.

3. Token-Based Auth (Best Modern Pattern)

Use JWT.

Token contains claims.

No session lookup needed.

Example

None

Authorization: Bearer JWT_TOKEN

Stateful Components in Real Systems

Even stateless architectures still need some stateful layers:

- Databases
- Message queues
- Search indexes
- File storage
- Session stores

So goal is usually:

Keep compute layer stateless, isolate stateful layers.

Writes Are Harder to Scale

Reads can be scaled via:

- Cache
- Replicas
- CDN

Writes to shared storage are harder because of:

- Locking
- Consistency
- Replication lag
- Transactions
- Conflicts

Techniques for Scaling Stateful Storage

1. Replication

Primary + replicas.

2. Sharding

Split data across nodes.

3. Compression

Reduce storage/network usage.

4. Aggregation

Batch writes.

5. Denormalization

Reduce joins.

6. Metadata Service

Store routing/index info separately.

Real Examples

Amazon

Frontend app servers stateless.

Cart/user data in distributed stores.

WhatsApp

Real-time messaging includes stateful session/routing layers.

Google Search

Search frontends stateless, indexes stateful.

Interview Answer Strategy

When asked to design scalable system:

Say:

Application Layer

Use stateless microservices.

Data Layer

Use stateful storage with replication/sharding.

Session Layer

Use Redis/JWT.

Final Summary

Stateless

Best for:

- APIs
- Web apps
- Microservices
- Scalable systems

Stateful

Needed for:

- Databases
- Real-time systems
- Transactions
- Persistent sessions